TEAM 9 • PES UNIVERSITY

SOFTWARE ARCHITECTURE & DESIGN SPECIFICATION

Electronics Store Inventory Management System

Updated architecture, components, interfaces and workflow design

Document control	Details
Document ID	EIMS-SAD-1.1
Version	1.1 — Sprint 2 enhancement revision
Date	03-11-2026
Authors	Team 9 — Karan Maheshwari, Krishitha Kankanala, Jayashree D, Hemavathi E
Status	Updated draft for review and evaluator sign-off
Repository	ElectronicsStore-ManagementSystem / Team-9-Electronics-Store-Inventory-Management-System

Revision purpose. Preserve the September 2026 baseline while extending the requirements, design and verification coverage to include functionality added during subsequent development sprints.

Revision history

Version	Date	Change summary	Approval
1.0	16-09-2026	Original layered architecture and baseline design.	Pending in source document
1.1	03-11-2026	Adds design coverage for supplier/branch management, image lifecycle, stock movements, purchase orders, availability monitoring, theme and API error handling.	For team/guide review

1. Introduction

1.1 Purpose. This document describes the EIMS v1.1 architecture and detailed design, extending the original baseline to cover the capabilities added during implementation.

1.2 Scope. The design covers browser UI, FastAPI REST API, authentication/RBAC, inventory, supplier, branch, stock movement, purchase order, reporting, monitoring, image handling, persistence and error handling. POS/payment-gateway integration remains outside scope.

1.3 Design principles. Layered separation; server-side authorization; transactional consistency for inventory changes; traceable stock operations; reusable validation/error handling; and testable module boundaries.

1.4 Implementation basis. Repository documentation identifies FastAPI, SQLAlchemy and SQLite in the current implementation. The exact production deployment topology and image-storage durability/backup configuration must be verified against the release environment.

2. Architecture overview

2.1 Architectural pattern. Layered architecture: browser presentation layer → FastAPI routing/API layer → application/service and authorization logic → repository/data-access layer → SQLAlchemy/SQLite persistence. This is a modular monolithic application, not a microservices system.

2.2 Runtime request flow. User action is sent from the browser to a REST endpoint. Authentication and authorization are checked before protected operations. Request data is validated, service logic applies business rules, repository operations read/write persistent entities, and the API returns a structured response. For stock changes, inventory quantity and movement history are committed consistently within a transaction.

2.3 Technology. Frontend: HTML/CSS/JavaScript as documented in the repository. Backend: Python/FastAPI. Persistence: SQLAlchemy with SQLite in the documented configuration. Tests: pytest and frontend JavaScript checks. CI workflow installs requirements and runs pytest on pushes; the workflow definition should be checked if PR-triggered CI is required.

3. Component architecture

Component	Responsibility	Requirement coverage
Web UI	Inventory, supplier, branch, PO, movement, report and dashboard screens; form validation; theme preference; image preview.	F-001–F-032; NF-003; NF-006
REST API Layer	Routes requests, validates payloads, applies HTTP semantics and returns response schemas.	F-001–F-032; SR-001; SR-003; SR-006
Authentication & RBAC	Login/session handling and role checks for protected operations.	SR-001–SR-005
Inventory Module	Component CRUD, SKU rules, search, filters, sorting, quantity and reorder data.	F-001–F-014; F-027
Supplier Module	Supplier records and catalog retrieval for component/PO workflows.	F-023; F-027; F-029
Branch Module	Branch records and branch management operations.	F-024
Image Management	Image upload validation, association, preview, replacement and removal.	F-010; F-025
Stock Movement Module	Stock-in/out operations, movement metadata, history retrieval and transaction consistency.	F-028
Purchase Order Module	PO creation, line items, status, partial/full receiving, receipts and inventory updates.	F-029
Reporting Module	Availability classification, filters, exports and controlled report errors.	F-015–F-018; F-030
Dashboard Module	Inventory totals, stock value, charts, recent items and low-stock alerts.	F-019–F-022
Monitoring Module	Monthly system availability calculations using monitoring records and maintenance periods.	NF-002; F-030
Repository/Data Access	SQLAlchemy persistence, query encapsulation, transaction boundaries and data retrieval.	All persistent requirements
Audit/History Data	Persist deletion audit data, stock movement metadata and PO receipt history.	F-014; F-028; F-029

4. Data design

The logical data model includes the entities below. Existing repository model definitions are authoritative for exact field names and constraints; this document describes the logical relationships needed by the workflows.

Entity	Logical attributes / purpose	Relationships
User	Account identity, password hash, role and session-related data.	One user performs many stock movements and audit events.
Component	Name, SKU, category, quantity, unit price, supplier, reorder level, active state and image reference.	Belongs to a category; may reference supplier; has many movements and PO line items.
Category	Category identity and display information.	One category classifies many components.
Supplier	Supplier identity and catalog information.	One supplier may be associated with many components and purchase orders.
Branch	Branch identity and management attributes.	Branch-related inventory/operations depend on configured branch relationships where supported.
StockMovement	Component, movement type, quantity, reason, timestamp and responsible user.	Belongs to one component and one acting user; append-oriented history.
PurchaseOrder	Unique PO number, supplier, status, timestamps and relevant totals/metadata.	Belongs to supplier; contains one or more PO items and receipts.
PurchaseOrderItem	Component, ordered quantity and received quantity.	Belongs to one PO and references a component.

Entity	Logical attributes / purpose	Relationships
PurchaseOrderReceipt	Receipt event, received line quantities, timestamp and idempotency information.	Belongs to a PO; records partial/full receiving events.
AuditEvent	Actor, timestamp, affected component and deletion event details.	Supports traceability for deletion actions.
MonitoringRecord	Availability/health observation and timestamp/status fields used by reporting.	Feeds monthly availability calculations; exact schema follows implementation.

5. Key workflow designs

5.1 Stock-in workflow. Manager selects component, quantity and reason → UI sends request → API authenticates and authorizes actor → service validates positive quantity and component → transaction increments on-hand quantity and inserts StockMovement → transaction commits → response returns updated quantity and movement reference. On failure, rollback leaves both quantity and history unchanged.

5.2 Stock-out workflow. Manager selects component, quantity and reason → API validates positive quantity and available stock → transaction decrements quantity and inserts StockMovement → commit and return updated balance. Insufficient stock is rejected without changing inventory or history.

5.3 Purchase-order creation workflow. Authorized user selects supplier and one or more components/ordered quantities → API validates supplier and line items → service generates unique PO number and initial status → transaction persists PO and lines → response returns PO details.

5.4 Purchase-order receiving workflow. Authorized user submits receipt quantities → API validates PO state and outstanding quantities → service checks idempotency → transaction creates receipt record, increments inventory for received quantities and records corresponding stock movement/history as implemented → status is recalculated → commit. Duplicate receive request must not increment stock twice; any failure rolls back all related writes.

5.5 Supplier and branch workflow. Authorized user submits validated record → API enforces role policy → service persists record → catalog/list endpoint exposes the saved record to permitted forms and filters.

5.6 Image lifecycle workflow. UI selects file → API validates content type and size (documented limit 5 MB) → image is stored/associated → UI renders preview. Replacement updates the association; removal clears it according to repository behaviour. Storage cleanup and backup policy must be explicitly confirmed for production.

5.7 Availability reporting workflow. API receives report period/filter → reporting service reads inventory/monitoring data → applies classification, coverage and scheduled-maintenance rules → returns report or controlled error. Missing data must not be misrepresented as full availability.

6. Sequence diagrams (textual UML-style)

The following interaction sequences are included to make the added workflows explicit. They can be converted to UML sequence-diagram notation for the final submission.

6.1 Stock movement

Manager → Web UI: enter component, type, quantity, reason
Web UI → REST API: POST movement request
REST API → Auth/RBAC: authenticate + authorize
REST API → Movement Service: validate request
Movement Service → Repository: begin transaction; read component
Movement Service → Repository: update quantity + insert movement
Repository → Database: commit
REST API → Web UI: updated balance + movement result
Failure path: rollback; return controlled error; no partial write.

6.2 Purchase order receiving

Manager → Web UI: select PO and received quantities
Web UI → REST API: POST receive request with idempotency key
REST API → Auth/RBAC: authenticate + authorize
REST API → PO Service: validate PO state and outstanding quantities
PO Service → Repository: check idempotency and load PO lines
PO Service → Repository: create receipt; update inventory; record movement; update PO status
Repository → Database: commit transaction
REST API → Web UI: receipt details and updated PO
Duplicate/failure path: return prior result or rollback; do not double-increment stock.

6.3 Supplier/branch administration

Admin/Manager → Web UI: submit supplier/branch form
Web UI → REST API: POST create request
REST API → Auth/RBAC: authenticate + authorize
REST API → Validation: validate required fields
Service → Repository: persist entity
Repository → Database: commit
REST API → Web UI: created record or structured validation/authorization error.

6.4 Component image replacement

User → Web UI: choose replacement image
Web UI → REST API: upload image
REST API → Auth/RBAC: authenticate + authorize
Image Service → Validation: check format and size limit
Image Service → Storage/Repository: store new image and update component reference
REST API → Web UI: image reference/status
Failure path: reject invalid file; preserve previous valid image.

7. API design

The repository's docs/API.md is the source of truth for current endpoint paths and methods. The table below summarizes documented API surfaces; request/response schemas and authentication requirements must remain aligned with the live OpenAPI documentation at /docs.

Area	Method	Path	Purpose
Authentication	POST	/api/auth/login	Authenticate user and establish application session.
Inventory	GET	/api/inventory	List/search/filter inventory.
Inventory	GET/PATCH/DELETE	/api/inventory/{id}	Retrieve/update/delete component subject to role policy.
Image	POST	/api/inventory/{id}/image	Upload/associate component image.
Stock movements	POST/GET	/api/inventory/{id}/movements	Create stock movement and retrieve component movement history.
Dashboard	GET	/api/dashboard	Dashboard summary and chart data.
Branches	GET/POST	/api/branches	List/create branch records.
Suppliers	GET/POST	/api/suppliers	List/create supplier records.
Supplier catalog	GET	/api/suppliers/catalog	Retrieve supplier catalog for UI workflows.
Categories	GET	/api/categories	Retrieve categories.
Purchase orders	POST/GET	/api/purchase-orders	Create/list purchase orders.
Purchase order detail	GET	/api/purchase-orders/{id}	Retrieve purchase order detail.

Area	Method	Path	Purpose
Purchase order status	PATCH	/api/purchase-orders/{id}/status	Update order status subject to state rules.
Purchase order receive	POST	/api/purchase-orders/{id}/receive	Record partial/full receiving and update inventory.
Purchase order receipts	GET	/api/purchase-orders/{id}/receipts	Retrieve receipt history.
Availability report	GET	/api/reports/availability	Generate filtered availability report.
Availability export	GET	/api/reports/availability/export	Export report in supported formats.
System availability	GET	/api/reports/system-availability	Generate monthly system availability report.

8. Error handling, validation and transaction design

API errors shall use consistent HTTP status codes and a stable JSON error structure. Validation errors identify invalid fields without leaking internal stack traces. Unauthenticated requests return an authentication failure; authenticated but unauthorized requests return a forbidden response. Missing resources return not-found responses. Unexpected failures return a generic server error and are logged without secrets.

Stock movement and purchase-order receiving operations require transaction boundaries. All related database writes—inventory quantity, movement record, receipt record and PO status—must succeed or roll back together. Idempotency handling prevents repeated receipt requests from applying stock changes more than once.

Image uploads validate supported formats and maximum size. File names and client-supplied URLs must not be trusted. Error messages must not reveal server filesystem paths or sensitive configuration.

9. Security architecture

Authentication protects all restricted API operations. Authorization is checked server-side for each operation. Passwords are stored as salted one-way hashes. Session inactivity timeout follows SR-005. Production traffic uses HTTPS/TLS 1.2 or later. Input validation, safe query construction and output encoding mitigate injection and XSS. New endpoints for suppliers, branches, stock movements, purchase orders and image operations inherit the same security controls.

Historical movement and receipt records should be treated as audit evidence: ordinary users must not be able to alter or delete history unless an explicitly authorized administrative retention process is designed and documented.

10. Architecture decisions and constraints

A layered modular monolith is appropriate for the current academic project scale and avoids unnecessary distributed deployment complexity. SQLite is suitable for the documented local implementation, but concurrent multi-user production deployment, backup, file storage durability and database migration strategy require environment-specific review. The design does not claim horizontal scaling or production TLS merely from local test results.

11. Traceability

The updated architecture mapping is summarized below. Detailed verification links are maintained in the companion STP.

Requirement group	Architecture mapping
F-001–F-005	Inventory Module, Web UI, REST API, Repository/DB
F-006–F-010	Inventory + validation + image storage + Repository/DB
F-011–F-014	Inventory + Auth/RBAC + Audit
F-015–F-018	Reporting + Repository/DB + Web UI
F-019–F-022	Dashboard + REST API + Repository/DB
F-023	Supplier Module + Auth/RBAC + Repository/DB

Requirement group	Architecture mapping
F-024	Branch Module + Auth/RBAC + Repository/DB
F-025	Image Management + validation + storage
F-026–F-027	Web UI + Inventory/Supplier/Category APIs
F-028	Stock Movement Module + Inventory + transaction/repository
F-029	Purchase Order Module + Supplier + Inventory + Receipt/Movement persistence
F-030 / NF-002	Reporting + Monitoring + error handling
F-031	Web UI theme preference
F-032	REST API error-handling middleware/exception mapping
NF-001–NF-006	UI/API/DB boundaries, performance, usability, scale and responsive design
SR-001–SR-006	Authentication/RBAC, session, transport, validation and safe data access

12. Risks, deployment and open design confirmations

Risks: concurrent inventory updates; failed PO receipt leaving partial stock changes; missing or incomplete monitoring records; image storage loss; incorrect role configuration; database/API unavailability; and unverified deployment security. Mitigations include transactions, idempotency, controlled test data, backup verification, RBAC tests and environment-specific deployment checks.

Before approval, confirm exact role permissions for branch and supplier administration; whether branch is linked to each component, order or user; exact PO status transition rules; exact image storage location and cleanup/backup behaviour; monitoring data source and monthly availability formula; and the final endpoint schemas. Where implementation differs from this design, record the deviation rather than silently claiming conformance.

13. References

Original EIMS SRS v1.0 dated 06-09-2026; EIMS Software Test Plan v1.0 dated 16-09-2026; Team 9 SAD v1.0 dated 16-09-2026; repository docs/API.md; docs/ARCHITECTURE.md; source models, routes and tests; merged PRs for supplier/branch management, stock movement history, purchase orders, image management, availability reporting and UI/API enhancements.